

Azure Core Concepts

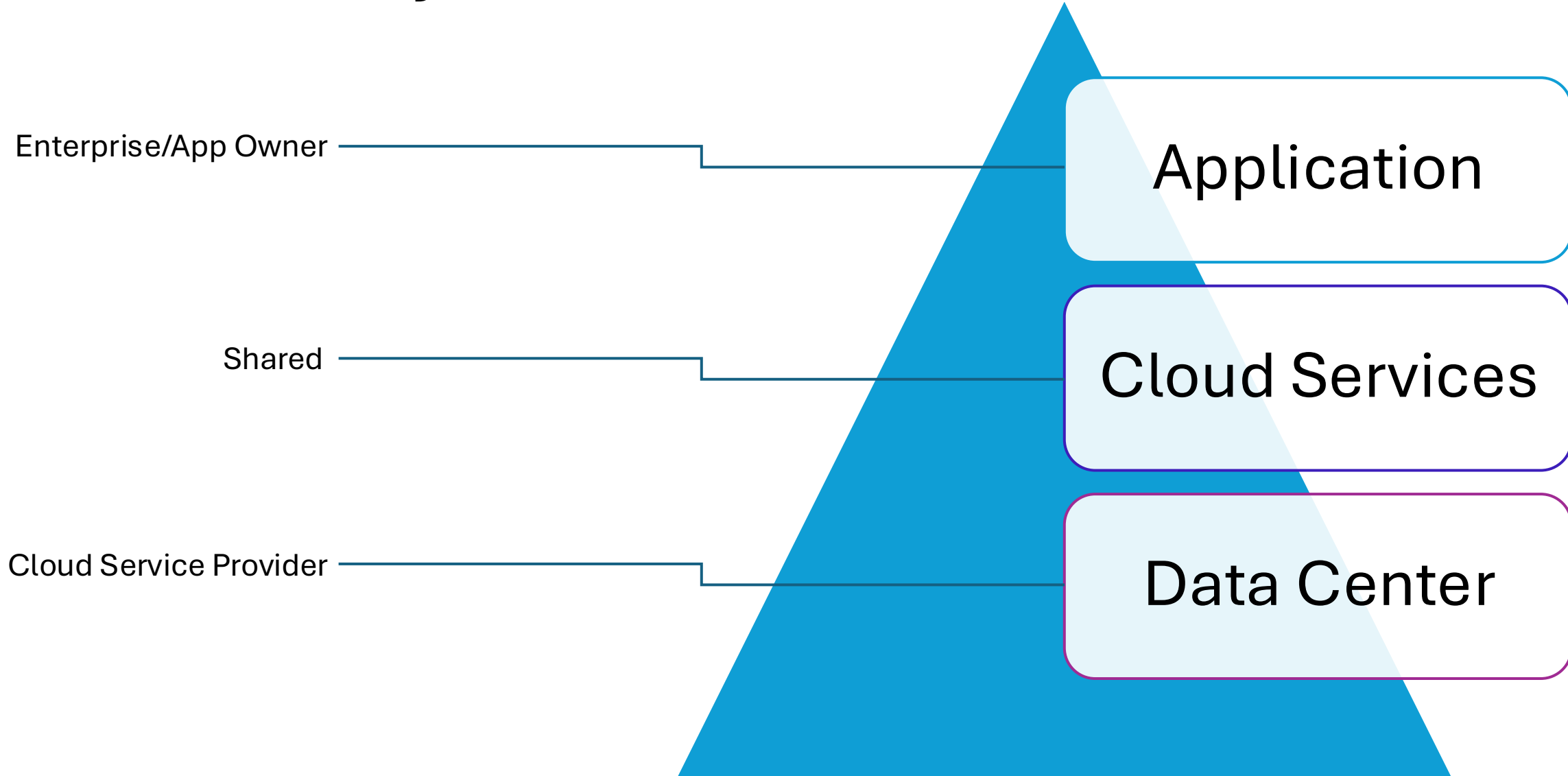
Core Concepts

- Region
- Management Group
- Subscription
- Resource Group
- Resources
- Azure Developer tools
 - Azure Portal
 - Azure REST API's
 - Azure SDKs
 - CloudShell – PowerShell / CLI
 - Azure ARM Template / Bicep Template
- Region Pairing
- Availability Scenario

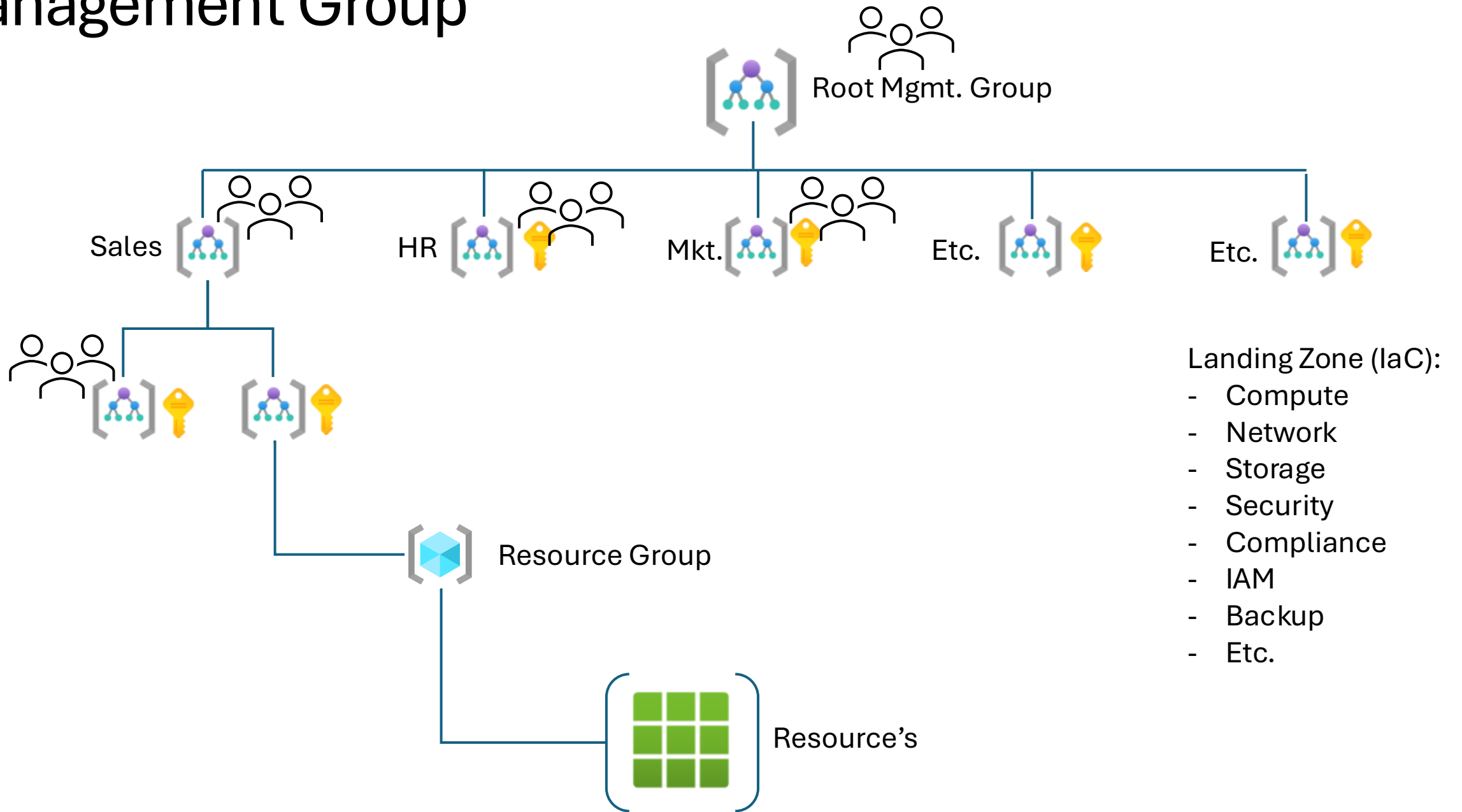
Region selection criteria

- Based on the user base location – to avoid latency
- Pricing
- Compliance
- Redundancy
- Service Avail.

Cloud Security Model



Management Group



Subscriptions

- Free Trial
- Pay – As – You – Go
- CSP
- EA
- MSDN

Resource Group

- Logical layer to organize resources in Azure
- Azure Services as resource provider
- Deploy RGs using IaC (Infra as a code)

Resources

- Azure Services
- Resource Provider – Microsoft.Compute, Microsoft.Storage

ARM Template / Bicep Template

- IaC tools – ARM Template / Bicep Template

Azure Operation Tools

- Portal
- SDKs
- CloudShell
- REST APIs
- ARM Template / Bicep Template

Region Pairing

- East US – West US
- West US – East US

Azure App Services

IaaS vs PaaS selection criteria

Virtual Machine



- First time migration in cloud
- Complete control in hand
- Legacy app/framework support
- Cloud agnostic approach

App Service

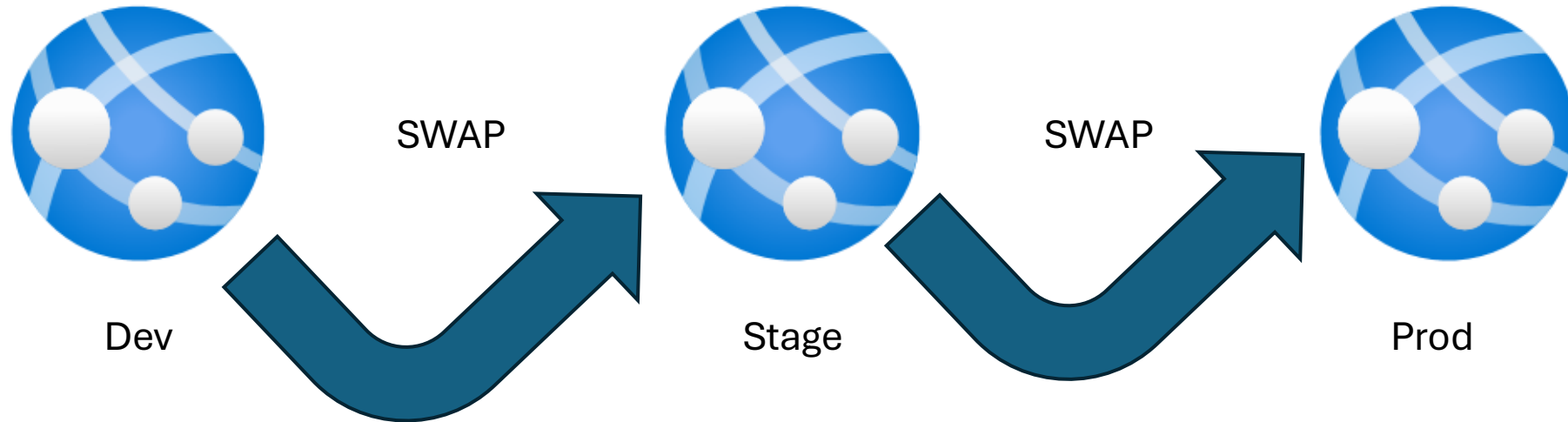


- Ease of use/operation
- Latest tech stack
- Global market deploy

App Service Plan

Features	Free	Shared	Basic	Standard	Premium	Isolated
Types of Apps	Code / Binary file-based app deployment, Container based Apps					
Number of Apps	10	100	Unlimited Apps			
Custom Domain Name	NA		Supported			
Auto Scaling	NA		Manual	Auto Scaling		
Deployment Center	GitHub, Azure DevOps, BitBucket, etc.					
Deployment Slots	NA			Supported		
Application Framework						

Deployment Slots



3 Tier App

3

Front End App



- Creating App Service – web app
- Mapping API endpoint as env. Variable
- Upload web app code using Kudu
- Test and validate app by uploading images

2

Back End API



- Creating App Services - API App
- Mapping storage account connection string and env. Variable
- Upload API app code using Kudu
- Capture API endpoint to map with Web app

1

Storage Account

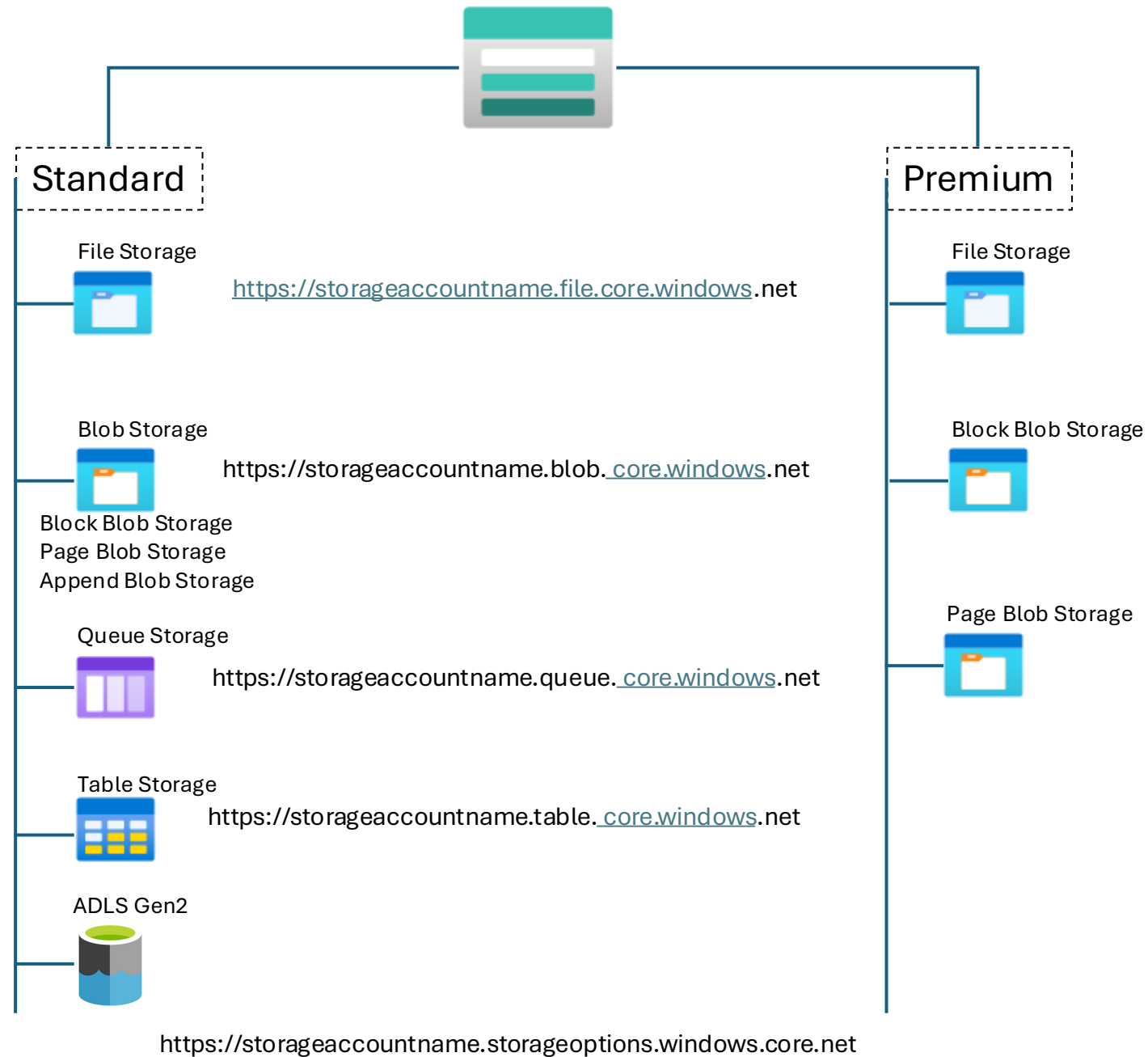


- Create a storage account
- Configure storage account with Blob Storage
- Creating Container storage
- Create container
- Upload first file manually for demo purpose
- Capture storage account connection string

Azure Storage

Structured, Semi-structured, Un-structured

Storage Account



Blob Storage

Block Blob

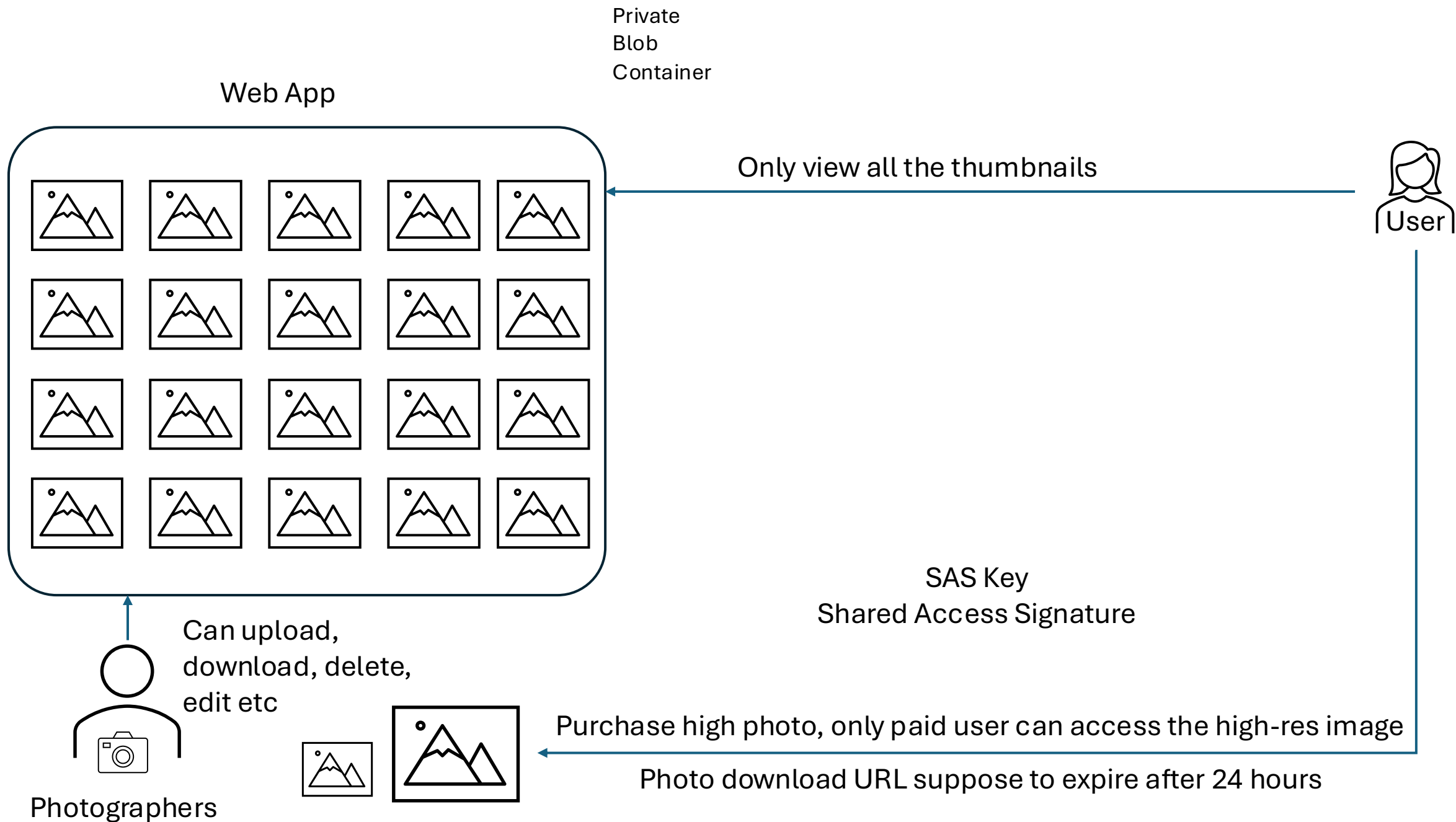
- Made up of block
- Allows you to store data in sequential order
- Best suitable for documents, images, video back up files etc.

Page Blob

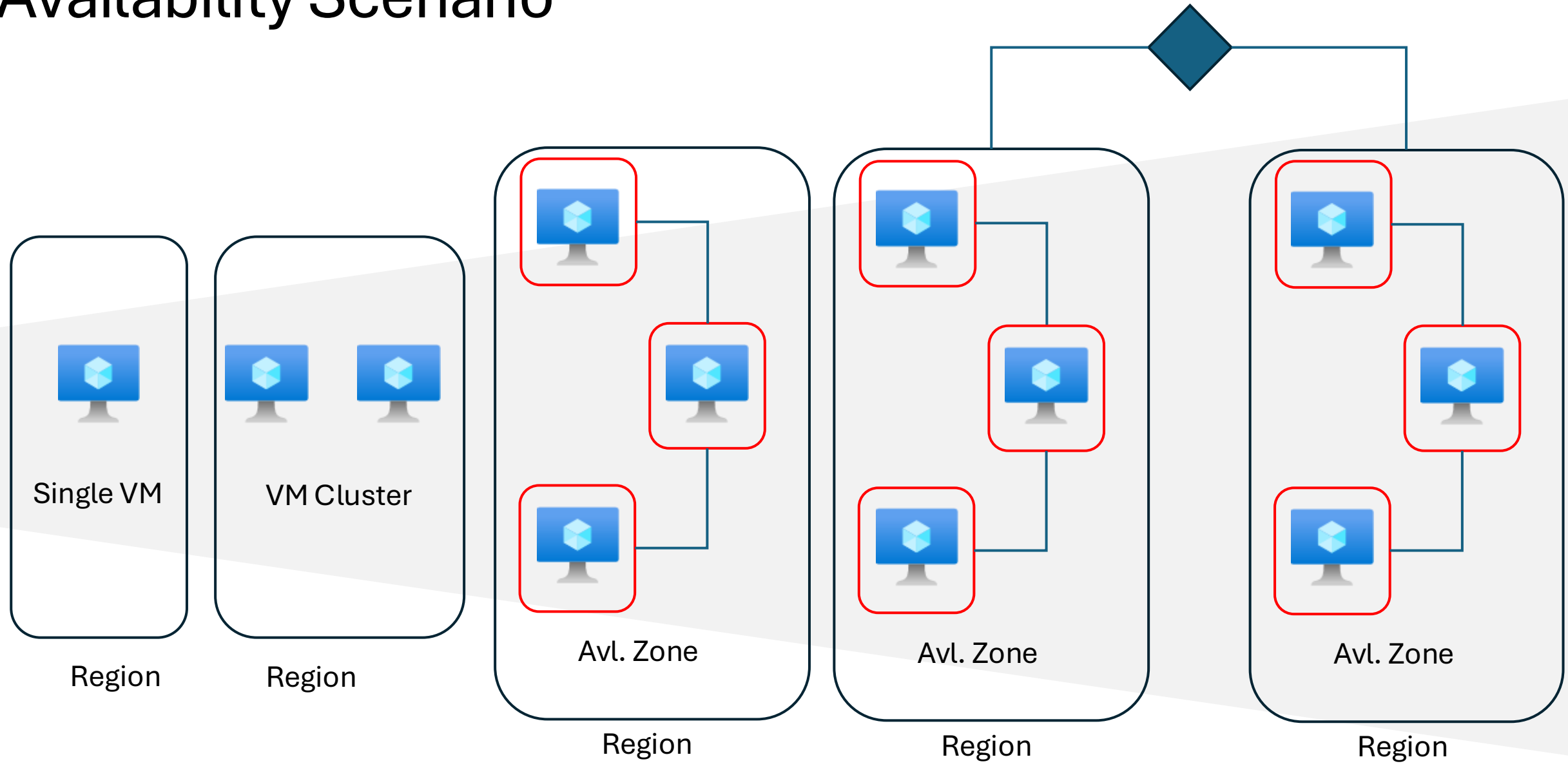
- Kind of disk storage
- Attach disk to VMs, container
- Random read/write operations
- HDD, SSD, Ultra SSD format

Append Blob

- Append blob
- Offers block blob features
- Log files, document which requires daily updates



Availability Scenario



Storage redundancy options

Regional

LRS

Locally Redundant Storage



- 3 copies of data into single region under single data center
- Protect against hardware failure, network
- Sync operation across the copy of data
- Best suitable for compliance kind of requirement
- Low-cost storage account

ZRS

Zone Redundant Storage

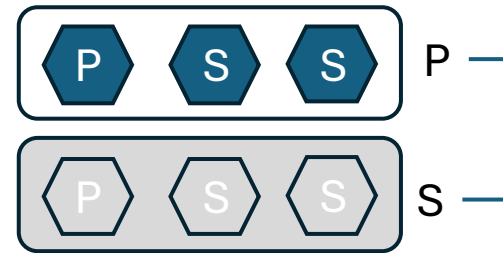


- 3 copies of data into single region under more than two data centers
- Protect against hardware, network and data center failure
- Sync operation across the copy of data

Global

GRS

Geo Redundant Storage



- 6 copies of data across two regions
- Protect against region level failure
- Async operation across the copy of data
- Better redundancy and SLA for business continuity scenario

GZRS

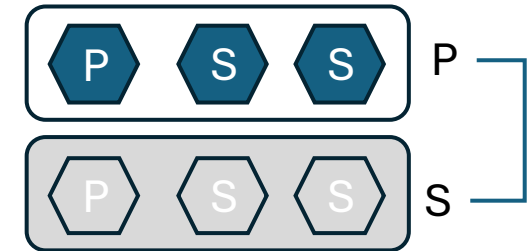
Geo Zone Redundant Storage

RA-GZRS

Read Access Geo Zone Redundant Storage

RA-GRS

Read Access Geo Redundant Storage



- GRS + Read access from secondary region

Azure Compute

Azure Compute

IaaS



VMs



VMSS

PaaS



App Service



Function Apps

CaaS



Container Instance



Web App For Container



Azure Container Apps



Azure Kubernetes Service

Function Apps

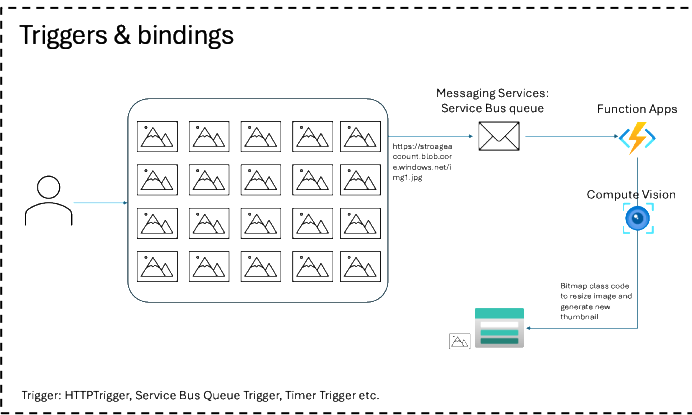
Function Apps



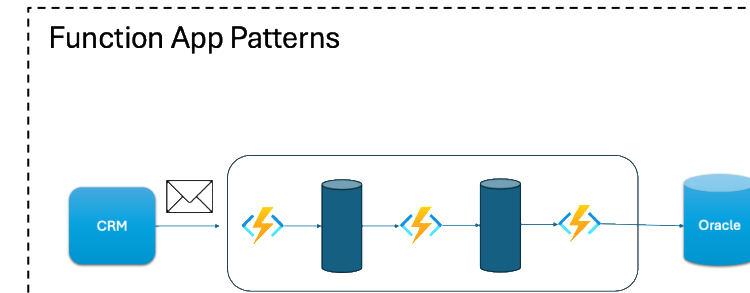
Triggers & Bindings

Hosting Plan

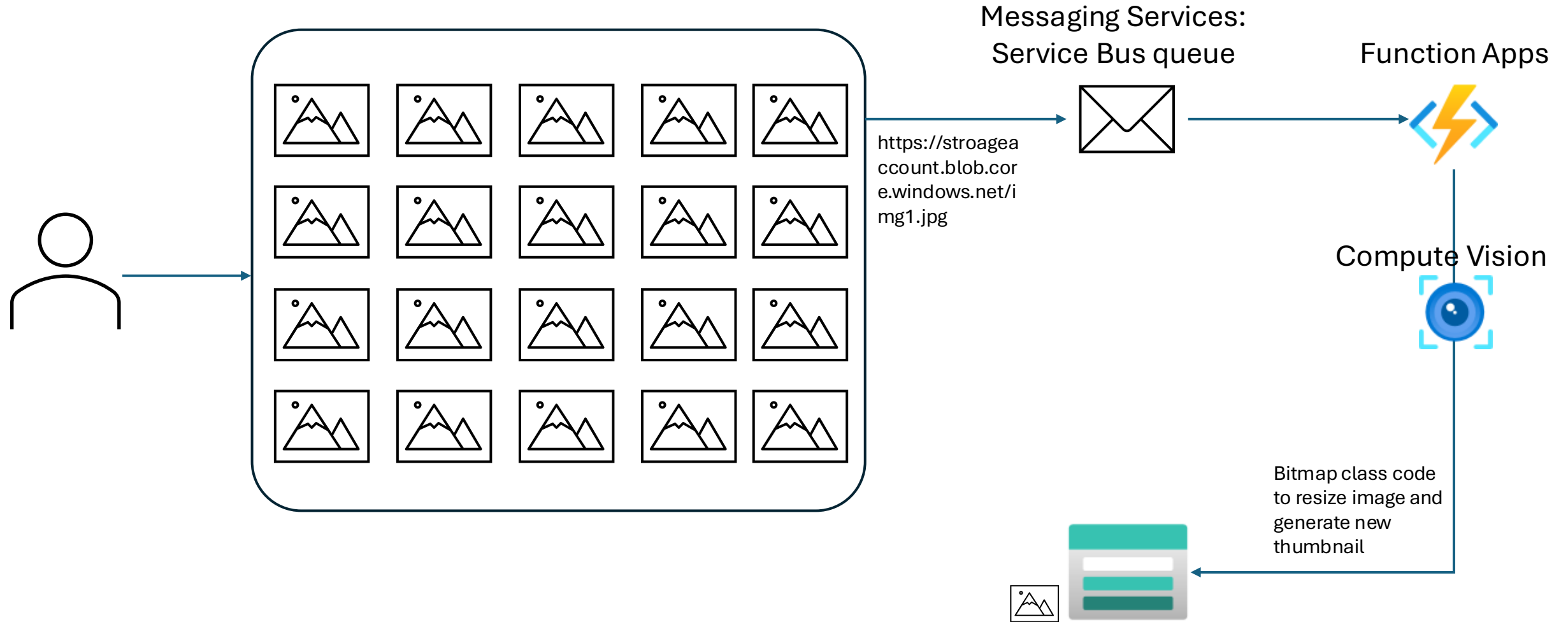
Function App Patterns



Hosting Plan					
Hosting plans	Flex Consumption	Functions Premium	App Service	Container Apps environment	Consumption (Windows)
	Get high scalability with compute choices, virtual networking, and pay-as-you-go billing.	Deploy multiple function apps on the same plan with event-driven scaling.	Run web apps and function apps on the same plan with more compute choices and pay for the instances of the plan.	Host function apps with other containerized microservices and pay for compute capacity.	Pay for compute resources when your functions are running (pay-as-you-go).
Scale to zero	✓	-	-	✓	✓
Scale behavior	Fast event-driven	Event-driven	Metrics based	Event-driven with KEDA	Event-driven
Virtual networking	✓	✓	✓	✓	-
Dedicated compute and prevent cold start	Optional with Always Ready	Minimum of 1 instance required	Minimum of 1 instance required	Optional with minimum replicas	-
Max scale out (instances)	1000	100	30	300	200



Triggers & bindings

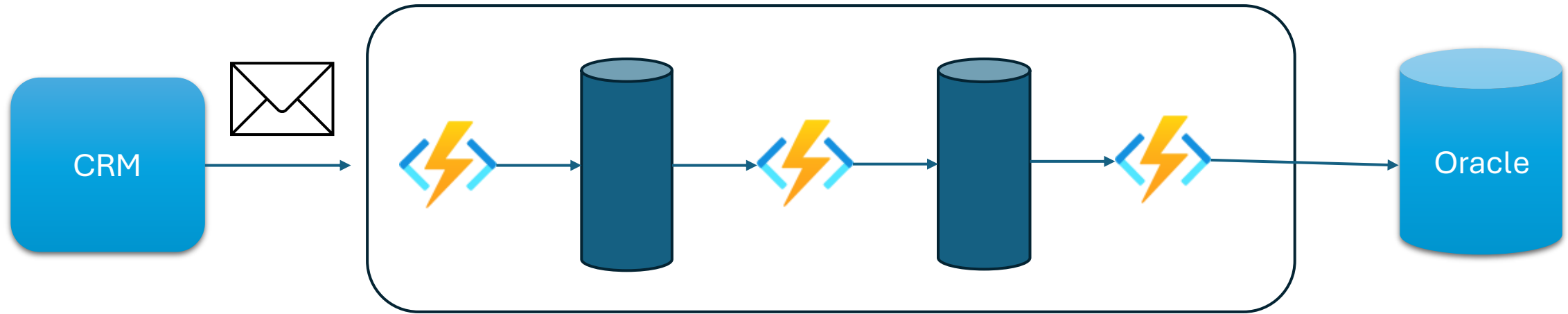


Trigger: HTTPTrigger, Service Bus Queue Trigger, Timer Trigger etc.

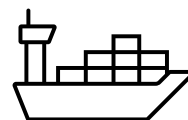
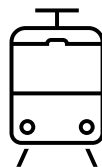
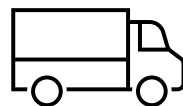
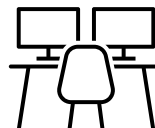
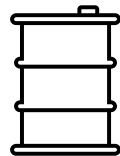
Hosting Plan

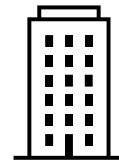
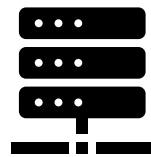
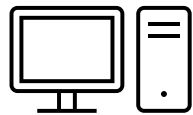
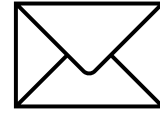
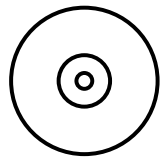
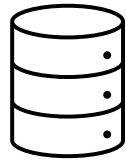
Hosting plans	☒ Flex Consumption Get high scalability with compute choices, virtual networking, and pay-as-you-go billing.	☐ Functions Premium Deploy multiple function apps on the same plan with event-driven scaling.	☐ App Service Run web apps and function apps on the same plan with more compute choices and pay for the instances of the plan.	☐ Container Apps environment Host function apps with other containerized microservices and pay for compute capacity.	☐ Consumption (Windows) Pay for compute resources when your functions are running (pay-as-you-go).
Scale to zero	✓	-	-	✓	✓
Scale behavior	Fast event-driven	Event-driven	Metrics based	Event-driven with KEDA	Event-driven
Virtual networking	✓	✓	✓	✓	-
Dedicated compute and prevent cold start	Optional with Always Ready	Minimum of 1 instance required	Minimum of 1 instance required	Optional with minimum replicas	-
Max scale out (instances)	1000	100	30	300	200

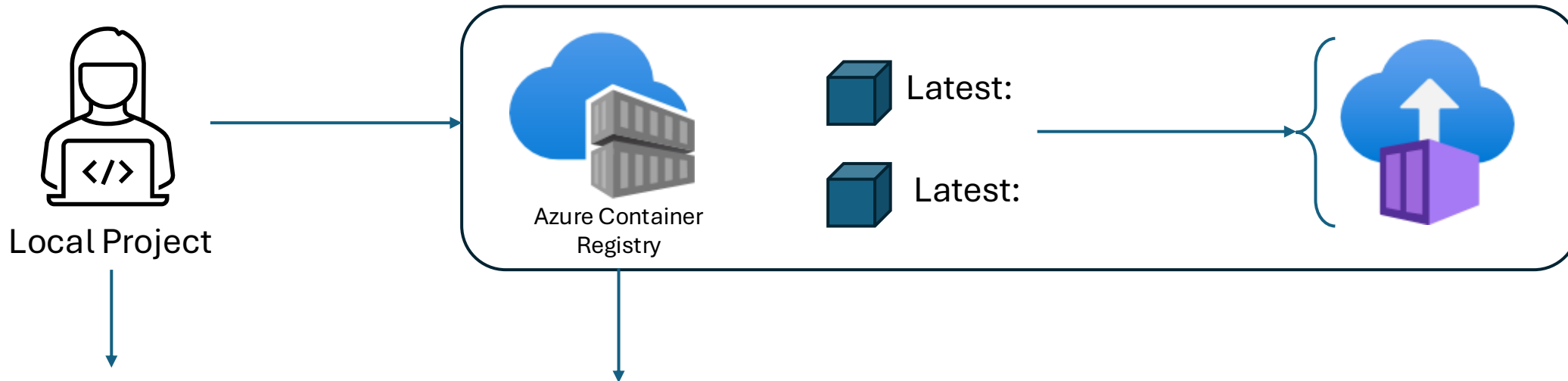
Function App Patterns



Azure Container Services



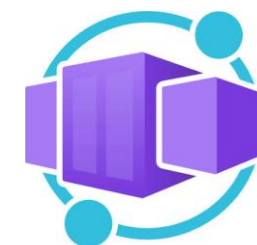




- Step 1: Create local project
- Step 2: Update the project with Dockerfile
- Step 3: Manage any dependencies if required and create project local zip file
- Step 4: Upload zip file to CloudShell and create container image and push that image to ACR

- Step 1: Create Azure Container Registry, using Azure CLI commands
- Step 2: Create and push container image in container registry
- Step 3: Under the access key of ACR, we need to enable admin user to deploy container image and access container image -

- Step 1: Create Azure Container Instance, using ACR deployment options
- Step 2: Test and validate if container working fine



Cosmos DB

Azure Cosmos DB

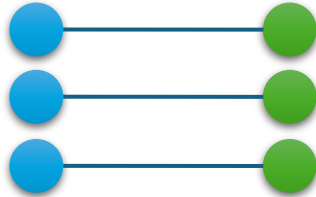


NoSQL DB APIs



Types of Data

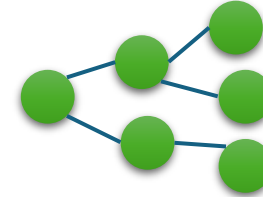
Key-Value



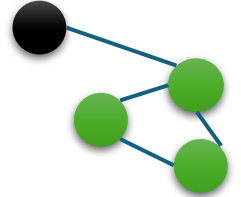
Column-Family



Documents



Graph



Cosmos DB Feature

Global
Distribution



Elastic Scale out

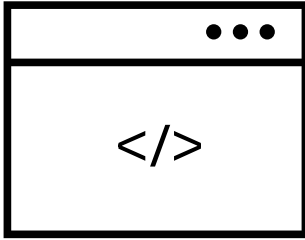


Five consistency
model



SLAs





Store Product Catalog list



Cosmos DB

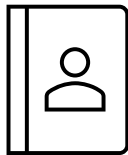
- SQL JSON DB
- Container
- Uploading item list via code

Store Product Images



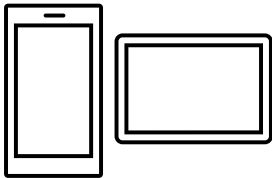
Identity Access Mgmt. / Entra ID & Security

Scenario



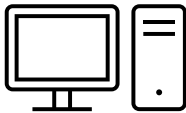
Users & Group

- Create and manage users & group in Entra ID
- Configure security groups
- Enable MFA – Multi Factor Authentication
- SSO –Single Sign On
- RBAC – Role Based Access Control
- Service Principle along with Managed Identity



Device Authentication / Authorization

- BYOD – Bring own device
- Microsoft InTune



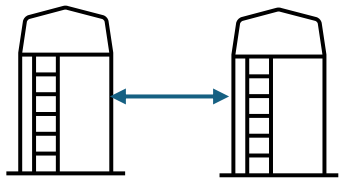
Domain Controller

- Domain Controller – Corporate identities



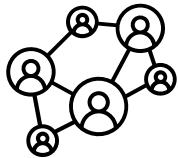
App Authentication / SaaS, Custom Apps

- Single tenant Apps
- Multi Tenant Apps



B2B

- Business to Business



B2C

- Business to Customer

Protocols: SAML, Oauth, OpenIDConnect etc..

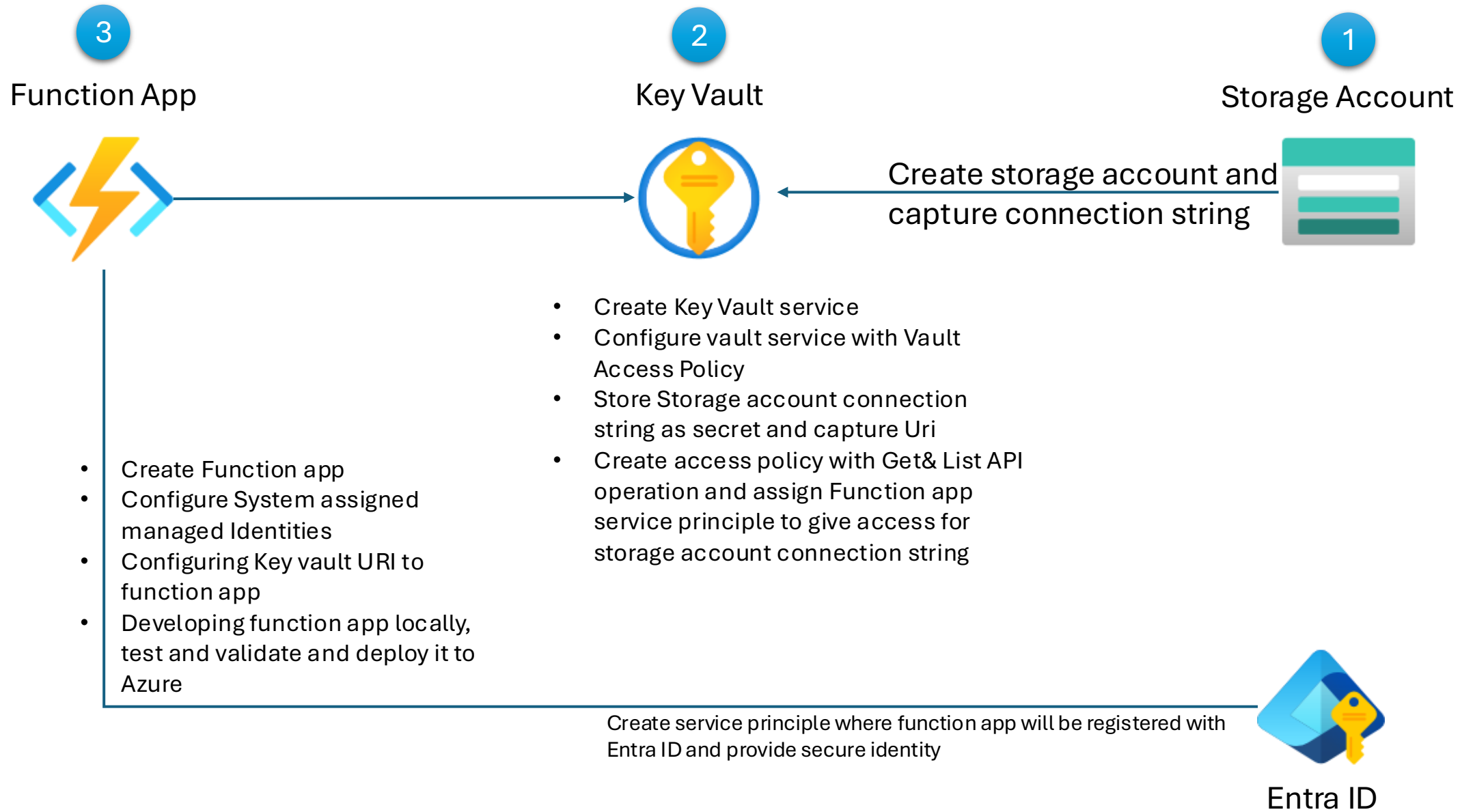
MSAL – Microsoft Auth. Lib.

RBAC – Role based access control

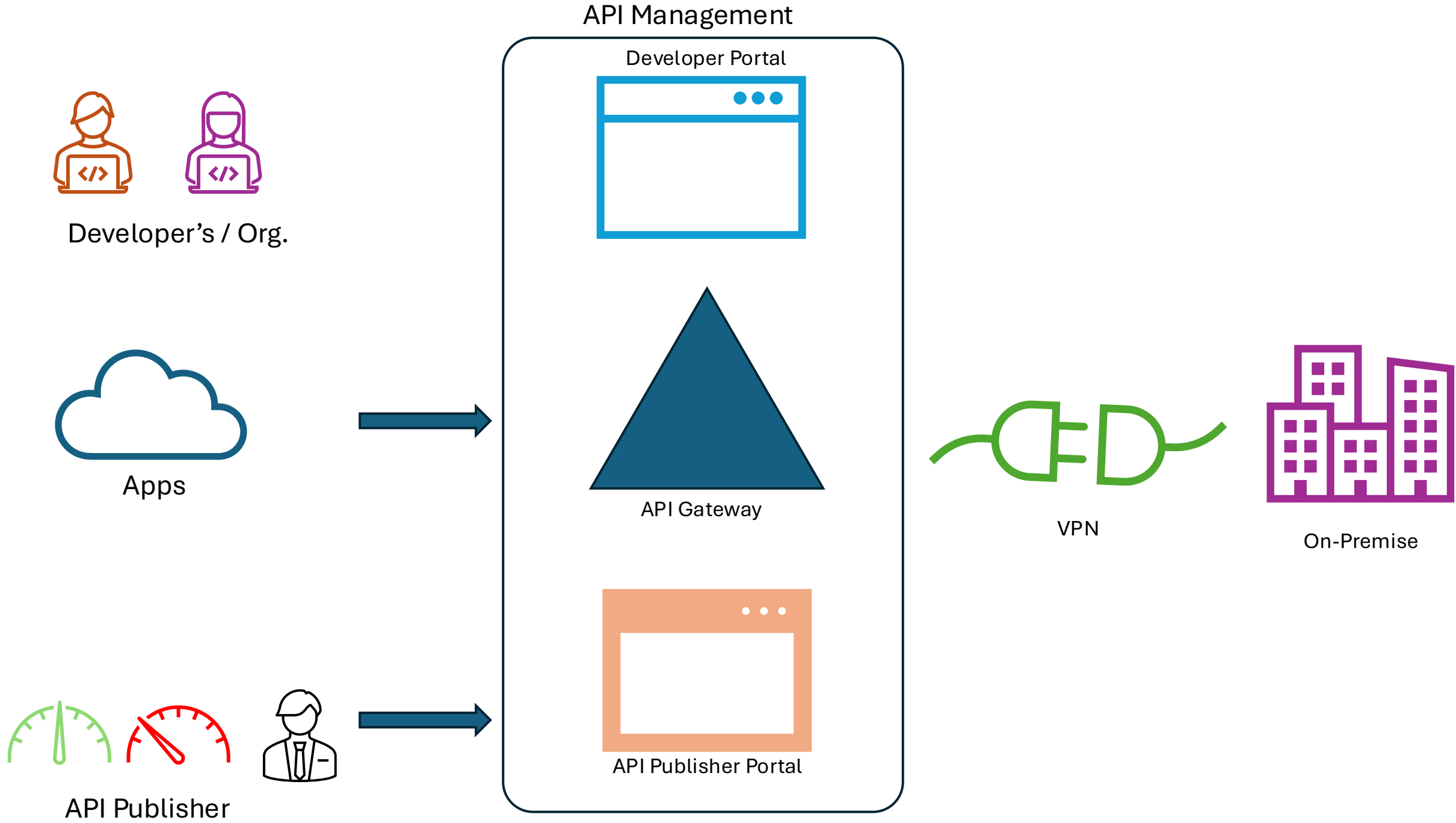
- Owner – full admin access to subscription and manage access to subscription. Delegate access to other user.
- Contributor – Full operational access except user delegation
- Reader – View access to services
- Custom – Allows you to customize the role

Managed Identities

- System assigned
- User defined managed identities



API Management



Messaging Service

Event Grid & Service Bus

Event Grid vs Event Hubs vs Service Bus

Features	Event Grid	Event Hubs	Service Bus
When to use?	React to status change	Telemetry and distributed data streaming	Order processing and financial transactions
Types of service	Event distribution	Event streaming	Message
Purpose	Reactive programming	Big data pipeline	High-value enterprise messaging

Event Grid

Event Publisher

Blob Storage
Resource Group
Azure Subscription
Event Hubs
Custom Topics

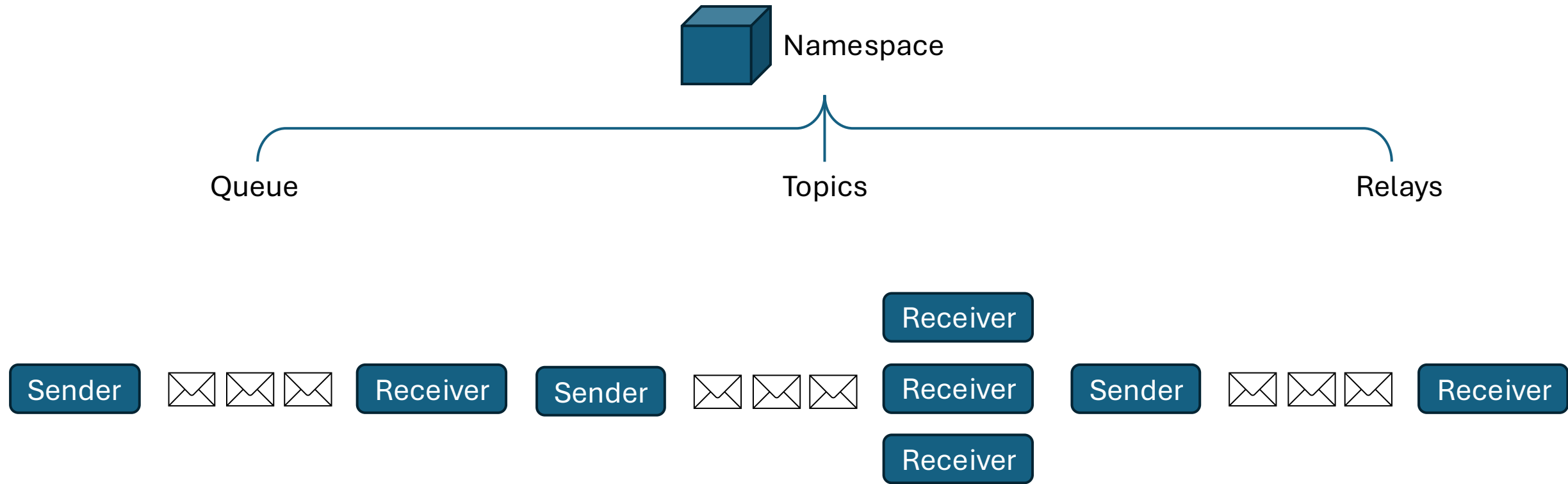


Event Grid

Event Handlers

Function App
Logic App
Azur Automation account
Web Hook

Service Bus



Azure Monitor



Azure Monitor

Monitoring Scenario

Application

Operating System

Azure Resources

Azure Tenant

Azure Subscription

Custom Resource



Insights: Application, Container, VMs, Storage etc.

Visualize: Dashboards, Views, PowerBI



Respond: Autos Scaling, Alert

Integrate: Logic App, Export API



App Insights

Analyze: Metrics Analysis, Log Analytics

KQL – Kusto Query Language

AI Lanscape

AI / Gen AI /
Agentic AI

 Foundry Portal

 VS Code





AI Capabilities

 Microsoft Foundry

 Microsoft Copilot Studio

Model Catalog

Foundation Model External Model Custom Model

Machine Learning

 Azure Machine Learning

 AML Studio

Data

SQL DBs
NoSQL DBs

 OneLake

 Dataverse

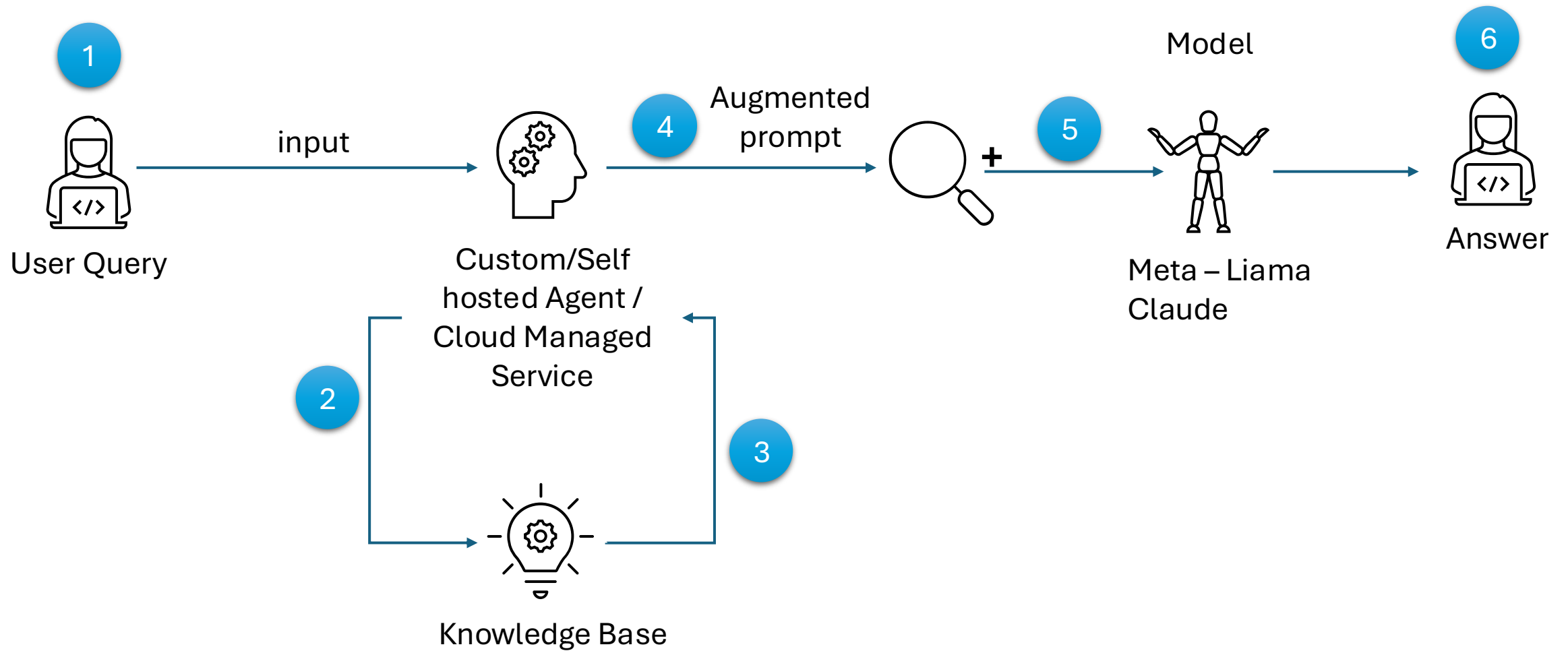
 Microsoft Fabric


Microsoft Purview

Infra

Compute (CPU, GPU), Network, Storage & Container services

AI Prompt Model Architecture



RAG

LLMs are powerful, but they operate in a vacuum.

Foundational models have incredible reasoning capabilities, but they are inherently limited by their training data. This creates critical business challenges:



Knowledge Cutoff

Models are unaware of events or data created after their training date, leading to outdated responses.



Lack of Proprietary Context

They cannot access your private, domain-specific data, making them generic.



Potential for Hallucination

When lacking a specific answer, models may invent facts, compromising accuracy.

Limitations

Knowledge Cutoff

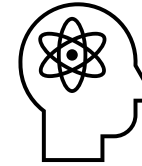
Hallucinations

Lack of Specificity

Generic response

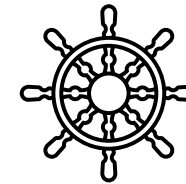
LLM

Two Layers solution stack



Knowledge Foundation:

Grounding LLM with verifiable, external data using RAG



The control layer (Prompt):

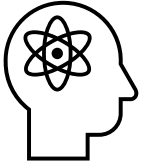
Precisely direct the LLM's behavior and output format using Prompt Engg.

RAG Use Case

- Q & A Chatbots
- Search Augmentation
- Content creation and summarization

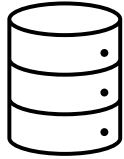


Core concepts of RAG



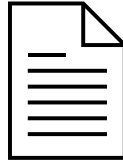
Index & Embed

- Embedding model which used for creating vector representation for documents and user queries



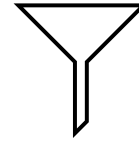
Vector Store

- Vector data can be stored with a Vector DB. Specialized to store unstructured data indexed by vector. E.g. AWS OpenSearch service, S3 etc..



Retrieval

- Searching stored vectors using similarity search to get relevant information



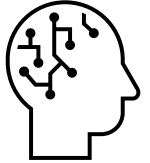
Filtering & Reranking

- Kind of process of selecting or ranking retrieved documents before passing as context



Prompt Augmentation

- Enhance the prompt workflow via injecting data retrieved from vector store



Generation

- We can use LLM for generating a response for the query

RAG Workflow

